- "Top DynamoDB Performance Metrics" by Jean-Mathieu Saponaro (https://www.datadoghq.com/blog/top-dynamodb-performance-metrics/)
- "Monitoring Elasticsearch With Sematext" (http://mng.bz/e1KV)

Next up, we'll cover two universal yet simple storage systems: file systems and object stores.

3.3.2 File systems and object stores

We will now move on to two storage-related sources that are widely used in cloud-native systems, typically for binary large objects (BLOB), such as images, videos, or documents (PDFs, Word documents, etc.). An important use case for local file systems in cloud-native contexts (e.g., Linux `ext4` or Window NTFS) is that of scratch space or, in other words, for managing temporary files. Networked file systems, such as NFS (https://datatracker.ietf.org/doc/rfc3530/), and object stores are oftentimes used for long-term storage, since they are in contrast to locally attached non-ephemeral storage (i.e., life cycles extend beyond the instance, container, or function existence).

The central place for all sorts of logs, from application logs to system logs, in Linux is the `/var/log` directory, and the dominant tooling for managing local logs is increasingly `journalctl`. I highlight the *local* here, since `journalctl` (by design) does not support shipping logs off a machine, leaving this as an exercise for the operator. We will revisit this issue in chapters 4 and 5, when we talk about log shipping from an agent and back-end destinations perspective.

From an infrastructure operator point of view (e.g., a Kubernetes platform admin), the most important file system signal is the amount of free space left on the root device, and various bad things can happen (https://sre.google/workbook/eliminating-toil/) if you run out of disk space. A simple way to check available disk space on a Linux system is the `df` (https://man7.org/linux/man-pages/man1/df.1.html) command. Take the following example:

Uses the -h option to print sizes in powers of 1024—that is, in a more human-readable format

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root       7.7G  3.7G  4.1G  47% /
tmpfs           3.8G     0  3.8G   0% /dev/shm
tmpfs           1.5G  904K  1.5G   1% /run
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
tmpfs           767M  4.0K  767M   1% /run/user/1000
```

We can see here that a little less than half of the disk space (47%) is used up on this machine.

You can gather further relevant file system metrics with low-level tooling, such as `iostat` and `nfsstat`, or using higher-level tools and agents, like Prometheus (https://prometheus.io/docs/guides/node-exporter/) or, in the case of OpenTelemetry, the Host Metrics receiver (http://mng.bz/OxRw).

For cloud-native applications, object stores are the workhorses, used for virtually anything from small files (JSON) to large BLOBs, such as images. Typically, these